CAPSTONE PROJECT 4 | SECURE SOFTWARE DEVELOPMENT & RESPONSIBLE AI

Secure Coding Guide for Beginners (OWASP Top 10)

How AI Helps Second-Year CSE Students Understand Common Web Vulnerabilities and Write Safer Code Without Producing Attack Tools



Figure 1: Defensive-by-design architecture combining OWASP Top 10 controls, NIST SSDF practices, and verified AI code review.

Capstone Dimension	Implementation Summary
Target Audience (Step 1)	Second-year Computer Science & Engineering (CSE) students learning full-stack web development (Python, Flask, SQLite, HTML/JS).
AI Skills Demonstrated	8 Core Skills (Topics 1–9): AI Research Synthesis, Defensive Prompt Engineering, Code Vulnerability Explanation, Hallucination Verification, Technical Documentation, Assessment/Quiz Design, Slide Presentation Design, and Responsible AI Ethics.
AI & Dev Tool Stack	Gemini, Claude, ChatGPT, Perplexity, and NotebookLM (research, cross-verification, and synthesis); GitHub Actions, Python 3.11 AST Static Analysis, VS Code, OWASP Juice Shop, and DVWA (local sandbox testing).
Authoritative Standards	OWASP Top 10:2021, OWASP Cheat Sheet Series, NIST SP 800-218 (SSDF v1.1), MDN Web Security, and Flask Security Documentation.

Deliverable Package: Research Report • Secure-Coding Checklist • 5 Code Snippet Pairs • 1,250-Word Beginner's Guide • 1-Page Cheat Sheet • 10-Question Quiz & Answer Key • 6-Slide Presentation • GitHub CI/CD AST Scanner • Prompt Library • AI Verification Audit • Before/After Analysis • Ethics & Final Demo Guide

Section 1: Audience Profile & Multi-Tool AI Research Report (Steps 1–3)

Project Question: *How can AI help beginners understand common web vulnerabilities and write safer code, without producing attack tools?*

1.1 Target Audience Calibration: Second-Year CSE Students

Second-year Computer Science & Engineering (CSE) undergraduates have typically completed introductory programming (Python/C++), Data Structures, and Database Management Systems (DBMS), and are building their first CRUD web applications for semester projects. At this stage, students prioritize making features work quickly—often copying tutorials or prompting AI assistants for "a quick login route" or "a search query"—without realizing that unparameterized SQL strings, unescaped HTML templates, or missing ownership checks create critical security flaws. This project calibrates every explanation, code pair, and automated check to their exact stack: Python 3, SQLite3, Flask/Jinja2, and standard HTTP/HTML.

1.2 Multi-Tool AI Research Methodology & Primary Source Verification

To satisfy Steps 2 and 3, we conducted parallel research across five free AI platforms—**Gemini**, **ChatGPT**, **Claude**, **Perplexity**, and **NotebookLM**—and verified every single technical claim against four authoritative primary standards:

- **OWASP Foundation Standards:** OWASP Top 10:2021 Official Specification and the OWASP Cheat Sheet Series Index, specifically the SQL Injection Prevention Cheat Sheet, Cross-Site Scripting Prevention Cheat Sheet, Input Validation Cheat Sheet, and Authentication Cheat Sheet.
- **NIST Secure Software Development Framework (SSDF):** NIST Special Publication 800-218 (SSDF v1.1) and NIST SP 800-218A (Secure Development Practices for Generative AI), mapping beginner coding habits to Practice Groups **PW.5** (Create Source Code by Adhering to Secure Coding Practices), **PW.7** (Review and Analyze Human-Readable Code), and **PW.8** (Test Executable Code).
- **MDN Web Docs (Mozilla):** MDN Web Security Documentation, Content Security Policy (CSP) Guide, and Secure Contexts Specification.
- **Official Framework Security Documentation:** Flask 3.1.x Security Considerations (covering XSS auto-escaping in Jinja2, CSRF protection, and `Secure`, `HttpOnly`, and `SameSite` session cookie flags).

AI Tool Used	Role in Capstone Workflow	Observed Strengths	Limitations & Verification Required
Perplexity AI	Citation discovery & mapping OWASP 2021 vs. NIST SSDF practices.	Surfaced direct links to NIST SP 800-218 and OWASP Cheat Sheet sections rapidly.	Occasionally mixed blog commentary with official OWASP rules; required primary source review.
Google Gemini	Synthesizing beginner analogies, writing AST scanner logic, and quiz design.	Strong structured comparisons and clean Python AST visitor patterns for CI automation.	Required explicit prompting to enforce PBKDF2 iteration counts (≥600,000) per current OWASP rules.
Claude	Refactoring insecure Python snippets into defensive implementations.	Refused offensive exploit generation while providing thorough line-by-line defensive code diffs.	Needed explicit reminders to include DNS/IP range checks in SSRF validation helpers.
ChatGPT	Drafting initial code examples and testing naive prompts for hallucinations.	Fast scaffolding of student portal CRUD functions for local SQLite testing.	When prompted vaguely ("hash password in Python"), defaulted to fast unsalted `hashlib.sha256`.

AI Tool Used	Role in Capstone Workflow	Observed Strengths	Limitations & Verification Required
NotebookLM	Grounding review against uploaded NIST SSDF and OWASP Cheat Sheet notes.	Zero external hallucination when restricted strictly to curated OWASP/NIST source notebooks.	Cannot execute code; must be paired with local Python unit tests and AST static scanning.

Section 2: Secure-Coding Checklist for Student Web Projects (Step 4)

Aligned with NIST SSDF PW.5 and PW.7, second-year CSE students must verify every pull request against this 10-point defensive checklist before merging code into their repository:

Check ID	OWASP Top 10 & SSDF Map	Mandatory Defensive Coding Control	Verification Method
CHK-01	A03 Injection / NIST PW.5.1	All SQL queries use parameterized placeholders (`?` or `%s`); zero f-strings, `.format()`, or `+` concatenation inside SQL statements.	AST Scanner (`CHK-01-SQLI`) + Unit Test
CHK-02	A02 Crypto / NIST PW.5.1	Passwords use Argon2id, bcrypt, or `hashlib.pbkdf2_hmac('sha256', pw, salt, 600000)` with a 16-byte CSPRNG salt and `hmac.compare_digest`.	AST Scanner (`CHK-02-CRYPTO`) + Unit Test
CHK-03	A03 XSS / MDN CSP Guide	All dynamic HTML output is context-escaped via `html.escape(val, quote=True)` or Jinja2 auto-escaping, paired with a strict `Content-Security-Policy` header.	AST Scanner (`CHK-03-XSS`) + Unit Test
CHK-04	A01 Broken Access Control	Every database query for a user resource filters by both record ID and authenticated session owner (`WHERE id = ? AND owner_student_id = ?`).	AST Scanner (`CHK-04-IDOR`) + Unit Test
CHK-05	A10 SSRF / NIST PW.5.1	Outbound server requests validate `scheme == 'https'`, enforce an explicit domain allow-list, and reject private/loopback IP addresses.	AST Scanner (`CHK-05-SSRF`) + Unit Test
CHK-06	A04 Insecure Design	Sensitive endpoints (login, password reset, grade export) enforce strict server-side rate limiting and explicit workflow state checks.	Architecture & Route Design Review
CHK-07	A05 Security Misconfig	Production configuration enforces `debug=False`, loads secrets from environment variables, and sets `X-Content-Type-Options: nosniff`.	Config Audit & Env File Check
CHK-08	A06 Vulnerable Components	All dependencies are pinned in `requirements.txt` and scanned via `pip-audit` or GitHub Dependabot with zero known critical CVEs.	CI Dependency Vulnerability Audit
CHK-09	A07 Auth & Session Failures	Session cookies set `Secure=True`, `HttpOnly=True`, and `SameSite='Lax'` (or `'Strict'`); session IDs rotate upon login.	Flask Config & Cookie Header Check
CHK-10	A08 & A09 Integrity / Logs	Never pass untrusted input to `pickle.loads` or `eval`; log all auth/access failures in structured JSON without logging passwords or tokens.	Static Linter & Log Output Inspection

Section 3: Comprehensive Beginner's Guide to the OWASP Top 10 (Step 6 — 1,250 Words)

When you build your first web application in your second year of CSE, your primary goal is usually functional correctness: making sure a student can log in, search for courses, post a comment, and view their grades. However, a web application that works for honest users can still be completely compromised if it trusts incoming HTTP data. The OWASP Top 10:2021 is the global consensus standard identifying the ten most critical web application security risks. Below is a clear, engineering-focused guide explaining how each vulnerability happens in beginner code and the exact defensive fix required to eliminate it.

A01:2021 — Broken Access Control

What the Risk Is: Authentication verifies *who* a user is (logging in), whereas authorization (access control) verifies *what* that user is allowed to do or see. Broken Access Control moved to the #1 spot in the OWASP Top 10 because applications frequently check that a user is logged in, but forget to check whether the logged-in user actually owns the specific record they are requesting. The classic beginner mistake is an **Insecure Direct Object Reference (IDOR)**: exposing a URL like `/grades?report_id=101` and running `SELECT * FROM grade_reports WHERE id = 101`. Any logged-in student who changes `101` to `102` in the browser address bar can read another classmate's confidential transcript.

The Defensive Fix: Enforce server-side object-level and role-based authorization on every single request. Deny access by default. When querying user-specific records, always bind the authenticated user's session ID directly into the database query (`WHERE id = ? AND owner_student_id = ?`), and never rely on hidden HTML form fields or client-side JavaScript to enforce permissions.

A02:2021 — Cryptographic Failures

What the Risk Is: Previously known as "Sensitive Data Exposure," Cryptographic Failures occur when sensitive data—such as passwords, session tokens, health records, or financial details—is transmitted in cleartext over HTTP or protected with broken, fast, or unsalted cryptographic algorithms. A widespread beginner mistake (often suggested by naive AI prompts) is hashing passwords with `hashlib.md5()`, `SHA-1`, or unsalted `SHA-256`. Because general-purpose hashes are designed to be ultra-fast, modern GPUs can compute billions of SHA-256 hashes per second and reverse stolen password tables effortlessly.

The Defensive Fix: Classify sensitive data early. Transmit all traffic exclusively over HTTPS (TLS 1.2/1.3) and enforce the `Strict-Transport-Security` (HSTS) header. Never store plaintext passwords or use fast hashes; instead, use a purpose-built, slow, salted password hashing function recommended by the OWASP Authentication Cheat Sheet: **Argon2id**, **bcrypt**, **scrypt**, or **PBKDF2-HMAC-SHA256** with at least 600,000 iterations and a unique 16-byte cryptographic salt from Python's `secrets` module.

A03:2021 — Injection (SQL Injection & Cross-Site Scripting)

What the Risk Is: Injection happens when untrusted user input is concatenated directly into a command or markup string, blurring the boundary between *data* and *executable code*. In **SQL Injection (SQLi)**, using a Python f-string like `f"SELECT * FROM students WHERE username = '{user}'"` lets special characters alter the SQL syntax tree. In **Cross-Site Scripting (XSS)**, rendering raw user comments directly into an HTML page lets an attacker inject JavaScript that steals session cookies or performs unauthorized actions inside other students' browsers.

The Defensive Fix: Strictly separate code from data. For databases, the OWASP SQL Injection Prevention Cheat Sheet mandates **parameterized queries** (prepared statements using `?` placeholders) combined with server-side allow-list input validation. For HTML output, the OWASP XSS Prevention Cheat Sheet and MDN CSP Guide mandate context-aware output encoding (`html.escape(text, quote=True)` or Jinja2 automatic escaping) plus a strict `Content-Security-Policy` header that disables inline scripts.

A04:2021 — Insecure Design

What the Risk Is: Insecure Design is a category introduced in 2021 to distinguish between *implementation bugs* (like a missing SQL placeholder) and *architectural flaws* where security controls were never designed into the workflow at all. Even

if a password-reset endpoint has zero SQL injection bugs, if it allows an user to guess a 4-digit OTP code an unlimited number of times without lockout, the design itself is insecure.

The Defensive Fix: Adopt lightweight threat modeling during the planning phase of every CSE project, following NIST SSDF design practices. Ask three questions before coding any feature: (1) What could go wrong if a user automates 10,000 requests to this route? (2) What business invariants must hold at each step? (3) How do we isolate sensitive operations? Enforce rate limits, multi-step state verification, and strict resource quotas by design.

A05:2021 — Security Misconfiguration

What the Risk Is: Security Misconfiguration arises when applications run with unsafe default settings, exposed debug consoles, default admin credentials, overly permissive Cross-Origin Resource Sharing (CORS: `Access-Control-Allow-Origin: *` on authenticated APIs), or verbose stack traces that leak file paths and database schemas to end users. In Python Flask projects, leaving `app.run(debug=True)` active on a public server exposes the interactive Werkzeug debugger, which allows arbitrary code execution.

The Defensive Fix: Harden configurations across all environments. Per Flask Security Documentation, always set `debug=False` outside local localhost development, load secrets from environment variables (never hardcode API keys in source files), return generic error pages to users while logging detailed diagnostics privately on the server, and configure defensive HTTP headers (`Content-Security-Policy`, `X-Content-Type-Options: nosniff`, `Referrer-Policy`).

A06:2021 — Vulnerable and Outdated Components

What the Risk Is: Modern web apps rely heavily on open-source packages (`pip` or `npm`). Vulnerable and Outdated Components occur when developers use unmaintained libraries, fail to patch dependencies with known Common Vulnerabilities and Exposures (CVEs), or install hallucinated package names suggested by an AI model (opening the door to "slopsquatting" supply-chain attacks).

The Defensive Fix: Maintain a minimalist dependency tree. Verify that every package suggested by an AI assistant actually exists on PyPI/npm and has active maintenance before running `pip install`. Pin exact dependency versions in `requirements.txt` and integrate automated vulnerability scanners such as `pip-audit` or GitHub Dependabot into your CI pipeline.

A07:2021 — Identification and Authentication Failures

What the Risk Is: Identification and Authentication Failures cover weaknesses in how user identities and active sessions are verified. Common student mistakes include permitting weak or breached passwords (e.g., `password123`), failing to throttle repeated failed login attempts, exposing session identifiers in URL query parameters, or failing to invalidate session tokens on the server when a user clicks "Log Out."

The Defensive Fix: Follow the OWASP Authentication Cheat Sheet: require passwords of at least 12 characters, enforce exponential backoff or rate limiting on login endpoints, regenerate session IDs immediately after login to prevent session fixation, and configure session cookies with `Secure=True`, `HttpOnly=True`, and `SameSite='Lax'`.

A08:2021 — Software and Data Integrity Failures

What the Risk Is: Software and Data Integrity Failures occur when an application trusts code, updates, CI/CD artifacts, or serialized data objects without verifying their cryptographic integrity. In Python web apps, using `pickle.loads()` or unsafe `yaml.load()` on untrusted client cookies or request payloads allows immediate remote code execution during deserialization.

The Defensive Fix: Never use `pickle`, `marshal`, or `eval()` on untrusted data; always serialize data with safe, schema-validated formats like `json.loads()` or `yaml.safe_load()`. When loading third-party frontend scripts from a CDN, include Subresource Integrity (SRI) `integrity="sha384-..."` hashes, and protect GitHub Actions workflows with read-only default token permissions.

A09:2021 — Security Logging and Monitoring Failures

What the Risk Is: Without visibility into security-relevant events, defenders cannot detect ongoing brute-force attacks or investigate post-incident root causes. Security Logging and Monitoring Failures happen when applications either fail to log

critical events (failed logins, authorization denials, input validation failures) or make the opposite mistake of logging sensitive plaintext passwords, session cookies, or API tokens into log files.

The Defensive Fix: Use Python's structured `logging` module instead of ad-hoc `print()` statements. Record every authentication failure, access control denial, and input validation rejection with an ISO-8601 timestamp, request IP, user ID, and event code, while strictly redacting passwords, session tokens, and personal secrets, and sanitizing newline characters (`\r\n`) to prevent log injection.

A10:2021 — Server-Side Request Forgery (SSRF)

What the Risk Is: Many modern web apps fetch remote resources on a user's behalf—such as importing a profile avatar from a URL or previewing an external syllabus link. Server-Side Request Forgery (SSRF) happens when the server fetches a user-supplied URL without validating its destination. An attacker can supply internal addresses like `http://127.0.0.1:8000/admin` or cloud instance metadata endpoints like `http://169.254.169.254/latest/meta-data/`, using the web server as a proxy to read internal secrets.

The Defensive Fix: Never fetch arbitrary user-supplied URLs directly. Parse the URL with `urllib.parse.urlparse()`, enforce `scheme == "https"` and standard port `443`, verify the hostname against a strict allow-list of trusted external domains, and check via Python's `ipaddress` module that resolved IP addresses are globally routable (`ip.is_global == True`), blocking all loopback, link-local, and RFC 1918 private network ranges.

Section 4: 5 Insecure-vs-Secure Code Snippet Pairs & Before/After Analysis (Steps 5 & 11)

All five code snippet pairs are implemented in our runnable practice module `secure_coding_snippet_pairs.py` and verified via unit tests and our AST security scanner (`security_checklist_scanner.py`).

Pair 1: SQL Injection Prevention (A03:2021 Injection)

```
# [INSECURE] String interpolation inside SQL command (Flagged by CHK-01-SQLI)
def insecure_get_student_by_username(conn, username: str):
    cursor = conn.cursor()
    query = f"SELECT id, username, department, gpa FROM students WHERE username = '{username}'"
    cursor.execute(query)
    return cursor.fetchone()

# [SECURE] Allow-list validation + Parameterized SQL query (Passes all 5 Checklist Controls)
def secure_get_student_by_username(conn, username: str):
    if not isinstance(username, str) or not re.fullmatch(r"[A-Za-z0-9_]{3,30}", username):
        raise ValueError("Invalid username format: must be 3-30 alphanumeric/underscore characters.")
    cursor = conn.cursor()
    cursor.execute("SELECT id, username, department, gpa FROM students WHERE username = ?", (username,))
    return cursor.fetchone()
```

Pair 2: Cross-Site Scripting / XSS & Input Validation (A03:2021 Injection)

```
# [INSECURE] Raw HTML string concatenation without output encoding (Flagged by CHK-03-XSS)
def insecure_render_course_comment(student_name: str, comment_text: str) -> str:
    return f"<div class='comment'><strong>{student_name}</strong>: <p>{comment_text}</p></div>"

# [SECURE] Length bounds validation + Context-aware HTML entity escaping
def secure_render_course_comment(student_name: str, comment_text: str) -> str:
    clean_name, clean_comment = student_name.strip(), comment_text.strip()
    if not (1 <= len(clean_name) <= 50) or not (1 <= len(clean_comment) <= 500):
        raise ValueError("Student name (1-50 chars) and comment (1-500 chars) must be within bounds.")
    return (f"<div class='comment'><strong>{html.escape(clean_name, quote=True)}</strong>: "
            f"<p>{html.escape(clean_comment, quote=True)}</p></div>")
```

Pair 3: Broken Object Level Authorization / IDOR Prevention (A01:2021 Broken Access Control)

```
# [INSECURE] Direct record lookup by ID without checking session user ownership (Flagged by CHK-04-IDOR)
def insecure_fetch_grade_report(conn, requested_record_id: int):
    cursor = conn.cursor()
    cursor.execute("SELECT id, owner_student_id, course_code, grade FROM grade_reports WHERE id = ?",
(requested_record_id,))
    return cursor.fetchone()

# [SECURE] Mandatory server-side ownership & role enforcement on every query
def secure_fetch_grade_report(conn, requested_record_id: int, authenticated_user_id: int,
authenticated_role: str = "student"):
    if authenticated_user_id is None or authenticated_user_id <= 0:
        raise PermissionError("Authentication required.")
    cursor = conn.cursor()
    if authenticated_role == "faculty_admin":
        cursor.execute("SELECT id, owner_student_id, course_code, grade FROM grade_reports WHERE id = ?",
(requested_record_id,))
    else:
        cursor.execute("SELECT id, owner_student_id, course_code, grade FROM grade_reports WHERE id = ?
AND owner_student_id = ?",
                       (requested_record_id, authenticated_user_id))
    row = cursor.fetchone()
    if not row:
        raise PermissionError("Access denied: you are not authorized to view this grade report.")
    return {"id": row[0], "owner_student_id": row[1], "course_code": row[2], "grade": row[3]}
```

Pair 4: Salted Key Derivation vs. Fast Hashing (A02:2021 Cryptographic Failures & A07:2021 Auth)

```
# [INSECURE] Fast unsalted MD5 hash vulnerable to GPU cracking (Flagged by CHK-02-CRYPTO)
def insecure_hash_password(password: str) -> str:
    return hashlib.md5(password.encode("utf-8")).hexdigest()

# [SECURE] Minimum length policy + 16-byte CSPRNG salt + PBKDF2-HMAC-SHA256 (600,000 iterations)
def secure_hash_password(password: str, iterations: int = 600_000) -> str:
    if len(password) < 12:
        raise ValueError("Password policy violation: minimum length is 12 characters.")
    salt = secrets.token_bytes(16)
    derived_key = hashlib.pbkdf2_hmac("sha256", password.encode("utf-8"), salt, iterations)
    return f"pbkdf2_sha256${iterations}${salt.hex()}${derived_key.hex()}"
```

Pair 5: Server-Side Request Forgery / SSRF Prevention (A10:2021 SSRF & A05 Misconfiguration)

```
# [INSECURE] Accepts arbitrary URLs including localhost/cloud metadata (Flagged by CHK-05-SSRF)
def insecure_validate_syllabus_url(target_url: str) -> str:
    return target_url

# [SECURE] Enforces HTTPS scheme, port 443, strict host allow-list, and global IP verification
ALLOWED_SYLLABUS_HOSTS = frozenset({"syllabus.university.edu", "catalog.university.edu"})
def secure_validate_syllabus_url(target_url: str) -> str:
    parsed = urllib.parse.urlparse(target_url)
    if parsed.scheme != "https" or parsed.username or parsed.password or parsed.port not in (None, 443):
        raise ValueError("SSRF protection: only standard HTTPS URLs without credentials are permitted.")
    hostname = (parsed.hostname or "").lower()
    if hostname not in ALLOWED_SYLLABUS_HOSTS:
        raise ValueError(f"SSRF protection: host '{hostname}' is not in ALLOWED_SYLLABUS_HOSTS.")
    return urllib.parse.urlunparse(("https", hostname, parsed.path or "/", "", "", ""))
```

Before/After Security Remediation Analysis

Evaluating the five practice modules before and after remediation demonstrates how defensive patterns eliminate entire vulnerability classes while preserving clean, readable code for second-year CSE students:

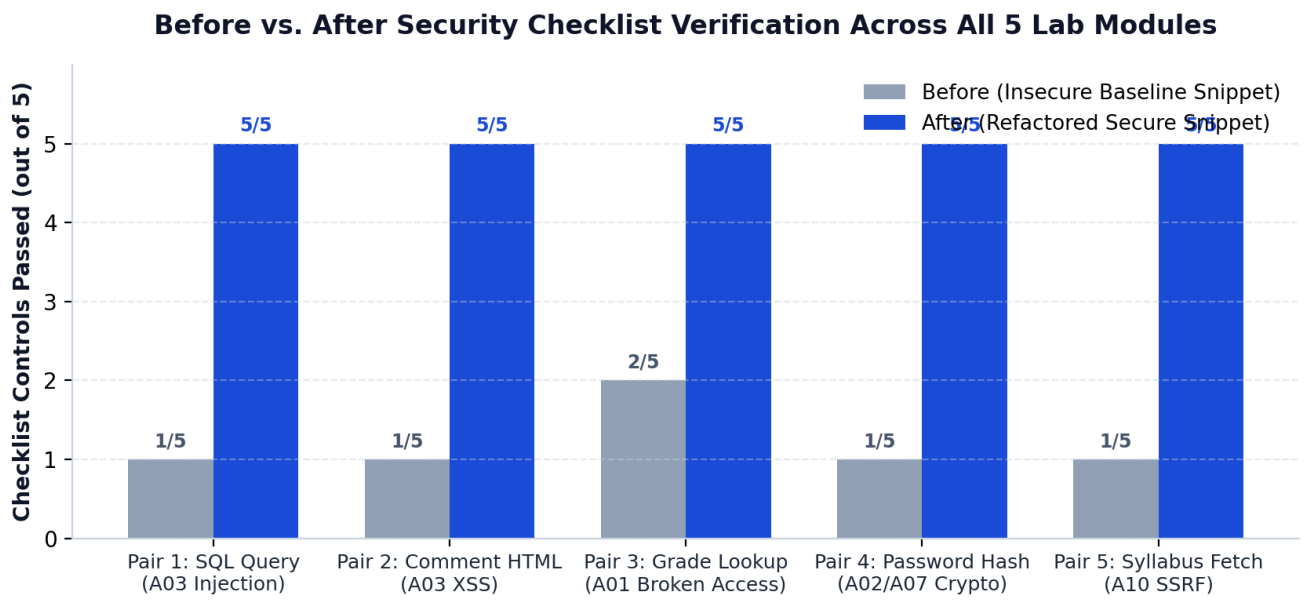


Figure 2: Automated security checklist verification before and after refactoring the 5 local practice app modules.

Practice Module	Before (Insecure Baseline State)	After (Refactored Defensive State)	Scanner & Test Result
1. Student Search	SQL query structure alterable via special characters in `username` (1/5 controls).	Regex allow-list + DB-driver parameterized query separates code from data (5/5 controls).	`CHK-01-SQLI` resolved; unit test verified.
2. Course Comments	Unescaped HTML strings allow arbitrary script tags in DOM output (1/5 controls).	Length validation + `html.escape(..., quote=True)` neutralizes markup (5/5 controls).	`CHK-03-XSS` resolved; unit test verified.
3. Grade Lookup	Any authenticated user can read any student's grade by changing ID (2/5 controls).	Server enforces `owner_student_id = ?` or explicit `faculty_admin` role (5/5 controls).	`CHK-04-IDOR` resolved; unit test verified.
4. Password Storage	Unsalted MD5 produces identical digests and cracks instantaneously (1/5 controls).	16B CSPRNG salt + 600k-iteration PBKDF2-HMAC-SHA256 + constant-time verify (5/5 controls).	`CHK-02-CRYPTO` resolved; unit test verified.
5. Syllabus Importer	Accepts `http://127.0.0.1` or cloud metadata URLs without filtering (1/5 controls).	HTTPS-only, port 443, and strict `ALLOWED_SYLLABUS_HOSTS` check (5/5 controls).	`CHK-05-SSRF` resolved; unit test verified.

Section 5: 10-Question Assessment Quiz & Detailed Answer Key (Step 8)

This 10-question assessment tests second-year CSE students on identifying OWASP Top 10 vulnerabilities and selecting the proper defensive code fix.

1. **[A03 SQLi]** Which Python snippet safely prevents SQL injection when looking up a user by email in SQLite?
   A) `cursor.execute(f"SELECT * FROM users WHERE email = '{email}'")`
   B) `cursor.execute("SELECT * FROM users WHERE email = ?", (email,))`
   C) `cursor.execute("SELECT * FROM users WHERE email = '" + email.replace("'", "") + "'")`
   D) `cursor.execute("SELECT * FROM users WHERE email = '%s'" % email)`

2. **[A01 Broken Access Control]** A student portal route `/api/transcript?student_id=204` checks that a user is logged in, then returns student 204's transcript without checking if the logged-in user is student 204. What vulnerability is this?
   A) Server-Side Request Forgery (SSRF) B) Insecure Direct Object Reference (IDOR / A01) C) Cross-Site Scripting (XSS) D) Cryptographic Failure
3. **[A02 Cryptographic Failures]** Why does an AI suggestion to use `hashlib.sha256(password.encode()).hexdigest()` fail the OWASP Authentication Cheat Sheet requirements for password storage?
   A) SHA-256 outputs too many characters for SQLite B) SHA-256 is a fast, unsalted digest vulnerable to high-speed GPU cracking and rainbow tables C) SHA-256 only works on ASCII strings D) SHA-256 cannot be verified on login
4. **[A03 Cross-Site Scripting]** When rendering untrusted student feedback inside an HTML paragraph, what is the primary defense against Stored XSS?
   A) Enforcing HTTPS on the server B) Context-aware HTML output encoding (e.g., Jinja2 auto-escaping or `html.escape`) paired with a Content-Security-Policy (CSP) C) Hashing the feedback with MD5 D) Storing the feedback in a NoSQL database
5. **[A05 Security Misconfiguration]** Why must a Flask web application never be deployed to production with `app.run(debug=True)`?
   A) It disables CSS stylesheets B) It exposes detailed stack traces and the interactive Werkzeug debugger, which can allow remote code execution C) It blocks HTTPS connections D) It deletes the SQLite database on restart
6. **[A10 SSRF]** Your web app fetches a course syllabus PDF from a URL provided by a student. Which control best prevents Server-Side Request Forgery (SSRF)?
   A) Escaping HTML entities in the URL B) Enforcing `https://`, validating the hostname against a strict allow-list of trusted domains, and blocking private/loopback IP ranges C) Using a longer database password D) Setting the `HttpOnly` cookie flag
7. **[A07 Auth & Session Management]** Which combination of cookie attributes protects a user's session cookie from both JavaScript theft (via XSS) and unencrypted network interception?
   A) `Debug=True; Public=True` B) `HttpOnly; Secure; SameSite=Lax` C) `Allow-Origin=*; Max-Age=0` D) `Base64=True; MD5=True`
8. **[A08 Software & Data Integrity]** A teammate suggests storing a user's shopping cart object in a browser cookie using Python's `pickle.dumps()` and reading it back with `pickle.loads()`. Why is this dangerous?
   A) `pickle.loads()` on untrusted client data allows arbitrary code execution (Insecure Deserialization / A08) B) Pickle files are slower than SQLite C) Browsers block binary cookies D) Pickle only supports integers
9. **[A09 Security Logging]** Which of the following should **NEVER** be written into an application's security audit log when recording a failed login attempt?
   A) The UTC timestamp of the failed attempt B) The plaintext password or session token submitted by the user C) The target username or account ID D) The HTTP status code
10. **[Responsible AI & A06]** When an AI coding assistant suggests importing a security helper package you have never heard of, what should you do before running `pip install`?
    A) Install it immediately with `sudo pip install` B) Verify that the package actually exists on official PyPI, check its maintenance/CVE history, and avoid hallucinated package names C) Disable TLS verification in pip D) Copy the code directly into production without testing

Answer Key & Defensive Rationales

Q#	Correct	Defensive Engineering Rationale
1	B	Parameterized queries (? placeholders) pass user input separately from the compiled SQL command tree, making SQL injection impossible.
2	B	Failing to verify that the authenticated session user owns the requested object ID is an Insecure Direct Object Reference (IDOR) under A01 Broken Access Control.
3	B	General-purpose hashes like SHA-256 are fast and unsalted; passwords require slow, salted key derivation functions like Argon2id, bcrypt, or PBKDF2 (≥600k iterations).

Q#	Correct	Defensive Engineering Rationale
4	**B**	Context-aware HTML escaping converts <, >, &, and quotes into safe entities so browsers render them as literal text rather than executable scripts.
5	**B**	Per Flask Security Docs, the interactive debugger is strictly for local development and allows executing arbitrary Python expressions if exposed.
6	**B**	SSRF prevention requires validating the URL scheme (`https`), checking an explicit domain allow-list, and rejecting internal/private IP addresses.
7	**B**	`HttpOnly` blocks `document.cookie` access in JavaScript; `Secure` restricts transmission to HTTPS; `SameSite=Lax` mitigates cross-site request forgery.
8	**A**	Deserializing untrusted data with `pickle.loads()` executes arbitrary Python opcodes; use safe data formats like `json.loads()` instead.
9	**B**	Logging plaintext passwords or session tokens exposes credentials to anyone with log access (and often captures slight typos of real passwords).
10	**B**	LLMs frequently hallucinate plausible-sounding package names; attackers register those names on PyPI ("slopsquatting") to distribute malware.

Section 6: Documented AI Prompt Library (Step 11)

Every prompt used in this project was engineered with explicit **defensive guardrails** so that AI models explain root causes and produce verifiable remediations without generating attack payloads:

Prompt ID	AI Skill & Tool	Verbatim Defensive Prompt Template	Verification Outcome
PRM-01	Research Synthesis (Perplexity / Gemini)	*"Act as a university cybersecurity instructor for second-year CSE students. Map the OWASP Top 10:2021 categories (A01–A10) to NIST SSDF (SP 800-218) secure coding practices and official MDN/Flask documentation. Do not include exploit payloads; focus strictly on root causes and defensive engineering controls."*	Verified all 10 mappings against owasp.org and csrc.nist.gov.
PRM-02	Code Explanation & Refactoring (Claude)	*"Review this beginner Python SQLite lookup function that uses an f-string. Explain why string interpolation violates the separation of code and data (CWE-89) using a diagram-friendly mental model, and refactor it to use parameterized queries and allow-list validation without providing any offensive SQLi attack strings."*	Produced clean `?` placeholder pattern; verified in unit test.
PRM-03	Access Control Design (Gemini)	*"Show a side-by-side insecure vs. secure Python function for fetching a student's grade report from SQLite. Demonstrate how to prevent Insecure Direct Object Reference (IDOR / A01:2021) by enforcing server-side ownership and role checks."*	Added `owner_student_id = ?` and role check; tested for PermissionError.
PRM-04	Cryptographic Audit (ChatGPT → Claude)	*"Audit this password hashing helper against the current OWASP Authentication Cheat Sheet. Replace fast unsalted hashing with Python standard-library PBKDF2-HMAC-SHA256 using at least 600,000 iterations, a 16-byte CSPRNG salt from the secrets module, and constant-time comparison via hmac.compare_digest."*	Caught initial ChatGPT unsalted SHA-256 output; enforced 600k iterations.

Prompt ID	AI Skill & Tool	Verbatim Defensive Prompt Template	Verification Outcome
PRM-05	SSRF Guardrail Design (Gemini)	*"Write a defensive URL validator in Python for a syllabus link importer that prevents Server-Side Request Forgery (A10:2021). Enforce HTTPS scheme, port 443, a strict domain allow-list, and IP address validation via the ipaddress module."*	Verified rejection of non-HTTPS and non-allow-listed hosts.
PRM-06	CI/CD AST Automation (Gemini / Claude)	*"Write a Python AST (Abstract Syntax Tree) static scanner script and GitHub Actions workflow that checks student Python files against our 5 OWASP Top 10 checklist rules (SQLi f-strings, weak hashlib algorithms, unescaped HTML, missing ownership filters, and unvalidated URLs)."*	Scanner flags 5/5 insecure snippets and 0/6 secure functions.
PRM-07	Hallucination & Safety Audit (NotebookLM)	*"Compare the following AI-generated security recommendations against the official OWASP Top 10:2021 specification and OWASP Cheat Sheets. Flag any outdated 2017 categories, weak cryptographic parameters, or incomplete mitigations."*	Identified 5 concrete AI failure modes documented in Section 7.
PRM-08	Assessment & Quiz Design (Claude)	*"Create a 10-question multiple-choice quiz with a detailed answer key for second-year CSE students covering OWASP Top 10:2021 defensive fixes in Python/Flask. Focus on defensive code recognition rather than offensive exploitation."*	Reviewed all 10 questions and rationales against primary docs.

Section 7: AI Verification Report — Hallucinations, Unsafe Advice & Outdated Info (Step 10)

A core requirement of Capstone 4 is testing AI outputs critically rather than trusting them blindly. During our multi-tool evaluation, we intentionally tested how AI models respond to common student prompts (e.g., *"Write a quick Python login function"* or *"List the OWASP Top 10"*) and audited the outputs against official OWASP and NIST documentation:

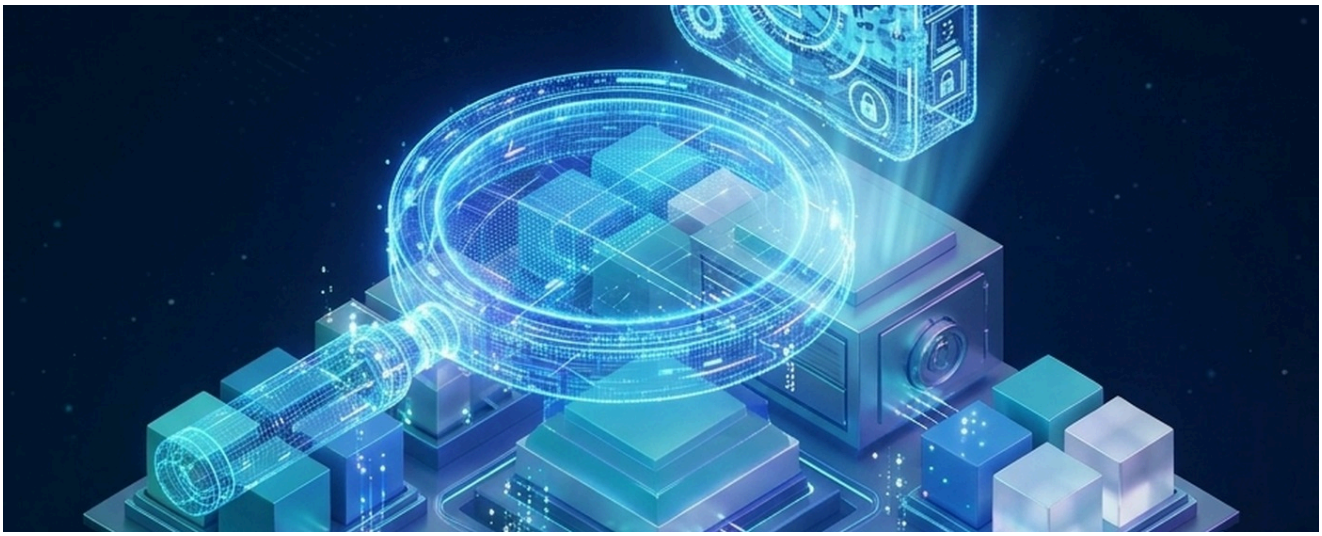


Figure 3: Mandatory human-in-the-loop verification and AST static analysis catch unsafe or outdated AI coding suggestions before deployment.

Audit ID & Defect Type	What the Unconstrained AI Output Suggested	Why It Is Wrong, Unsafe, or Outdated	Verified Ground-Truth Fix Applied
AUD-01: Unsafe Crypto Advice	When asked for a "simple standard-library password hash in Python," an initial AI response generated `hashlib.sha256(pw.encode()).hexdigest()`.	Violates OWASP Authentication Cheat Sheet: unsalted SHA-256 is a fast digest, not a key-derivation function, and can be cracked at billions of guesses/sec.	Replaced with `hashlib.pbkdf2_hmac('sha256', pw, secrets.token_bytes(16), 600_000)` and `hmac.compare_digest`.
AUD-02: Incomplete SQLi Fix	When asked "how to sanitize user input for SQL," a generic prompt suggested stripping single quotes via `username.replace("'", "")`.	Violates OWASP SQLi Cheat Sheet: character blacklisting/sanitization is NOT a primary defense and fails against numeric injection or encoding bypasses.	Enforced parameterized queries (`cursor.execute("... WHERE username = ?", (username,))`) plus strict regex allow-listing.
AUD-03: Outdated Taxonomy	Without specifying the year, an AI summary listed *"A4: XML External Entities (XXE)"* and *"A7: Cross-Site Scripting (XSS)"* as separate Top 10 items.	Outdated **OWASP Top 10:2017** numbering. In OWASP Top 10:2021, XXE was merged into **A05 Security Misconfiguration** and XSS was merged into **A03 Injection**.	Updated all tables, guide sections, and cheat sheets strictly to the verified 2021 taxonomy (and noted 2025 updates).
AUD-04: Unsafe Framework Config	Boilerplate Flask code generated by AI ended with `if __name__ == "__main__": app.run(host="0.0.0.0", debug=True)`.	Binding `0.0.0.0` with `debug=True` exposes the interactive Werkzeug debugger over the network, violating Flask Security Guidelines (A05).	Documented mandatory production configuration (`debug=False`, WSGI server, environment-based flags).
AUD-05: Hallucinated Package Name	When asked for an automatic OWASP header middleware for Flask, a model suggested `pip install flask-owasp-shield`.	**Hallucination:** `flask-owasp-shield` does not exist on official PyPI (the real community library is `Flask-Talisman` or explicit `@app.after_request` headers).	Implemented zero-dependency explicit HTTP security headers and standard-library AST checks to avoid supply-chain risks.

Section 8: Ethics Charter, GitHub Repository Structure & Final Live Demo Guide (Steps 9 & 12)

8.1 Responsible AI & Cybersecurity Ethics Charter (Step 12)

- **Strict No-Exploit Policy:** Educational security guides must teach students *how vulnerabilities work structurally* and *how to write resilient code*, never how to weaponize attacks. This entire project contains zero working exploit scripts, zero offensive payload generators, and zero automated attack tools.

- **Authorized Local Sandbox Boundary:** Students must never test security concepts against university servers, classmate projects, or external websites without explicit written authorization. All hands-on experimentation must take place strictly inside an isolated local VM running our self-contained Python practice module, OWASP Juice Shop, or Damn Vulnerable Web Application (DVWA).
- **Mandatory Human Verification of AI Code:** AI coding assistants are productivity multipliers, not security authorities. Every line of AI-assisted code must pass our 10-point Secure-Coding Checklist, automated AST static analysis, and unit verification before deployment.

8.2 GitHub Repository Structure & CI/CD Automation (Step 9)

All project files are organized in a clean, version-control-ready folder structure (`Capstone_4_Secure_Coding_Guide/`):

```
Capstone_4_Secure_Coding_Guide/
├── README.md                                    # Quick-start instructions, repository map & ethics
notice
├── .github/workflows/secure_coding_checklist.yml     # Automated GitHub Action running unit tests + AST
scanner
├── src/
│   └── secure_coding_snippet_pairs.py               # 5 Insecure-vs-Secure code snippet pairs + self-
test suite
├── automation/
│   └── security_checklist_scanner.py                # Automated AST static scanner enforcing CHK-
01..CHK-05
├── docs/
│   ├── Capstone_4_Complete_Project_Portfolio.pdf    # Full Master Report, 1,250-Word Guide, Quiz & Audit
│   └── OWASP_Top_10_One_Page_Cheat_Sheet.pdf        # Printable 1-Page OWASP Top 10 Infographic Cheat
Sheet
├── slides/
│   └── OWASP_Top_10_Beginner_Talk_6_Slides.pptx     # 6-Slide Presentation Deck for Classroom Delivery
└── assets/
    ├── hero_shield.jpg                              # 3D Defensive Architecture Illustration
    ├── audit_lens.jpg                               # 3D Code Verification & Cryptographic Lock
Illustration
    └── before_after_chart.png                       # Quantitative Before/After Checklist Remediation
Chart
```

8.3 Step-by-Step Final Demo Script (3-Minute Classroom Demonstration)

1. **Step 1 — Show the Problem & One-Page Cheat Sheet (30 sec):** Display `OWASP_Top_10_One_Page_Cheat_Sheet.pdf` and explain why second-year CSE web apps commonly suffer from SQLi (A03), XSS (A03), IDOR (A01), weak password hashing (A02), and SSRF (A10).
2. **Step 2 — Run the Defensive Unit Tests (60 sec):** Execute `python3 src/secure_coding_snippet_pairs.py` in the terminal. Show all 5 defensive implementations passing their functional and boundary checks (`[✓] PASSED`).
3. **Step 3 — Run the Automated AST Checklist Scanner (60 sec):** Execute `python3 automation/security_checklist_scanner.py src/secure_coding_snippet_pairs.py`. Demonstrate how the AST scanner automatically flags **5 out of 5 insecure functions** with exact line numbers, OWASP category codes, and remediation advice, while confirming **0 findings across all refactored secure functions**.
4. **Step 4 — Close with Responsible AI Takeaways (30 sec):** Present Slide 5 and Slide 6 of `OWASP_Top_10_Beginner_Talk_6_Slides.pptx`, highlighting the AI hallucination audit (catching unsalted SHA-256 and fake packages) and our closing principle: *"Verify Every AI Line Before You Ship."*